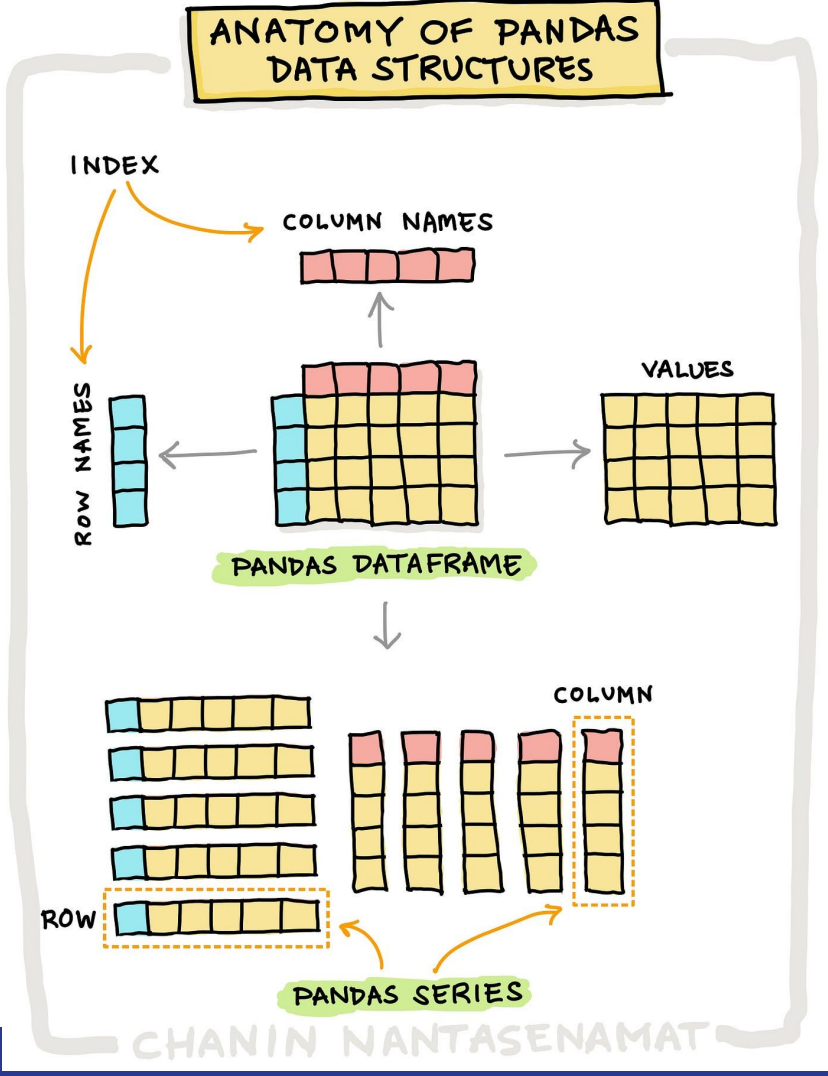
Pandas.DataFrame

Pandas.DataFrame

We can think of a DataFrame as a bunch of Series objects put together to share the same index



Create DataFrame from Series

Pass a **dict of Series** — keys become column names.

```
stu1 = pd.Series({"name": "John", "age": 20, "gender": "M"})
stu2 = pd.Series({"name": "Michael", "age": 24, "gender": "M"})
stu3 = pd.Series({"name": "Jessica", "age": 20, "gender": "F"})
pd.DataFrame([stu1, stu2, stu3], index=["s1", "s2", "s3"])
```

	name	age	gender
s1	John	20	M
s2	Michael	24	M
s3	Jessica	20	F

Create DataFrame from dict

Or a **dict of lists** — the most common way.

```
df = pd.DataFrame(
    {
        "name": ['John', 'Michael', 'Jessica'],
        "age": [20, 24, 20],
        "gender": ['M', 'M', 'F']
    }
    , index=["s1", "s2", "s3"])
df
```

	name	age	gender
s1	John	20	M
s2	Michael	24	M
s3	Jessica	20	F

Create Pandas.DataFrame from Row-Columned file

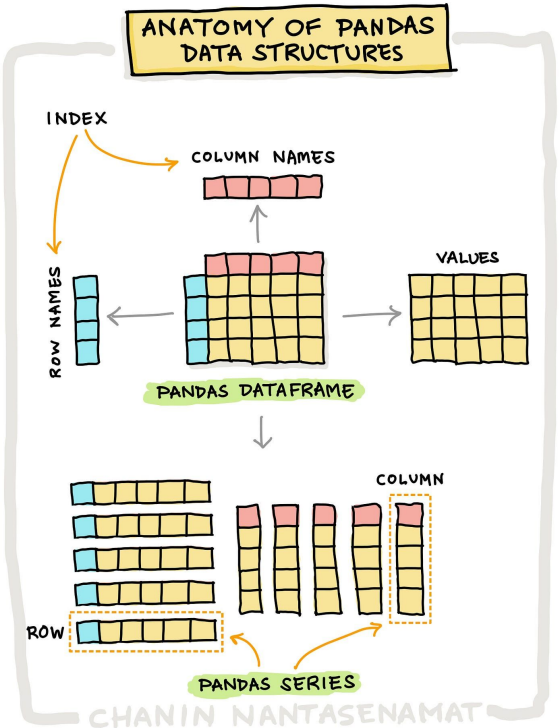
- pd.read_csv('file_path') loads a comma-separated file.
- pd.read_excel() / pd.read_json() handle other(excel, json) formats.

```
dataframe = pd.read_csv("./car-sales-extended-missing-data.csv")
dataframe
```

	Make	Colour	Odometer (KM)	Doors	Price
0	Honda	White	35431.0	4.0	15323.0
1	BMW	Blue	192714.0	5.0	19943.0
2	Honda	White	84714.0	4.0	28343.0
3	Toyota	White	154365.0	4.0	13434.0
4	Nissan	Blue	181577.0	3.0	14043.0
...	...	...	...	...	...
995	Toyota	Black	35820.0	4.0	32042.0
996	NaN	White	155144.0	3.0	5716.0
997	Nissan	Blue	66604.0	4.0	31570.0
998	Honda	White	215883.0	4.0	4001.0
999	Toyota	Blue	248360.0	4.0	12732.0

1000 rows × 5 columns

DataFrame Structure



Display DataFrame Props

```
dataframe.dtypes
```

```
Make              object
Colour            object
Odometer (KM)    float64
Doors            float64
Price            float64
dtype: object
```

```
dataframe.shape
```

```
(1000, 5)
```

```
dataframe.columns
```

```
Index(['Make', 'Colour', 'Odometer (KM)', 'Doors', 'Price'], dtype='object')
```

```
dataframe.index
```

```
RangeIndex(start=0, stop=1000, step=1)
```

Read / Query DataFrame

Getting a column

Cautious: this will return Series, not Dataframe

```
dataframe["Odometer (KM)"]
```

```
0     35431.0
1    192714.0
2     84714.0
3    154365.0
4    181577.0
Name: Odometer (KM), dtype: float64
```

```
dataframe.Make
```

```
0     Honda
1       BMW
2     Honda
3    Toyota
4    Nissan
Name: Make, dtype: object
```

Getting multiple columns

Cautious: this will return Dataframe

```
dataframe[["Odometer (KM)"]]
```

	Odometer (KM)
0	35431.0
1	192714.0
2	84714.0
3	154365.0
4	181577.0

```
dataframe[["Odometer (KM)", "Make"]]
```

	Odometer (KM)	Make
0	35431.0	Honda
1	192714.0	BMW
2	84714.0	Honda
3	154365.0	Toyota
4	181577.0	Nissan

Getting data by Label & Position

LABEL VS POSITION

Two ways to point at data.

- **loc** selects by **label** — the index and column names.
- **iloc** selects by **position** — integer row/column number.

```
df
```

	name	dept	age	salary	join_date
0	Sokha	Sales	28	1200	2021-03-15
1	Dara	IT	34	2100	2019-07-01
2	Mealea	Sales	25	1500	2022-01-20
3	Vibol	IT	41	2600	2018-11-05
4	Chan	HR	38	1800	2020-06-30

setup.py

```python
import pandas as pd, numpy as np

df = pd.DataFrame({
    "name":   ["Sokha", "Dara", "Mealea", "Vibol", "Chan"],
    "dept":   ["Sales", "IT", "Sales", "IT", "HR"],
    "age":    [28, 34, 25, 41, 38],
    "salary": [1200, 2100, 1500, 2600, 1800],
    "join_date": ["2021-03-15", "2019-07-01",
                  "2022-01-20", "2018-11-05", "2020-06-30"],
})
```

```python
df.loc[0, "name"]          # by LABEL    -> 'Sokha'
```

```
'Sokha'
```

```python
df.iloc[0, 0]              # by POSITION -> 'Sokha'
```

```
'Sokha'
```

Slicing by Position

ROW & COLUMN NUMBER · iloc

- **Syntax:** df.iloc[row_start:row_end:step]

- **Syntax:** df.iloc[row_start:row_end:step, col_start:col_stop:step]

- DataFrame supports slicing just like Python strings
- Rows only: df.iloc[2:4] — the column part is optional
- Columns only: df.iloc[:, 3:5] — a bare : keeps every row
- Positions start at 0 and the end is excluded, so iloc[2:4] returns rows 2 and 3

```
In [1]: df
```

Out[1]:

	name	dept	age	salary	join_date
0	Sokha	Sales	28	1200	2021-03-15
1	Dara	IT	34	2100	2019-07-01
2	Mealea	Sales	25	1500	2022-01-20
3	Vibol	IT	41	2600	2018-11-05
4	Chan	HR	38	1800	2020-06-30

```
In [2]: df.iloc[2:4]
```

Out[2]:

	name	dept	age	salary	join_date
2	Mealea	Sales	25	1500	2022-01-20
3	Vibol	IT	41	2600	2018-11-05

```
In [3]: df.iloc[:, 3:5]
```

Out[3]:

	salary	join_date
0	1200	2021-03-15
1	2100	2019-07-01
2	1500	2022-01-20
3	2600	2018-11-05
4	1800	2020-06-30

Slicing by Label

ROW & COLUMN KEY · loc

- **Syntax:** df.loc[row_start:row_end:step]

- **Syntax:** df.loc[row_start:row_end:step, col_start:col_stop:step]

- Uses the index labels and column names, not positions
- Slice with df.loc["Dara":"Vibol"] or pass a list of keys
- Pick rows and columns together: df.loc[["Dara", "Chan"], ["dept", "salary"]]
- Unlike iloc, loc includes the end label — "Vibol" is in the result

In [1]:
```
df = df.set_index("name")
df
```

Out[1]:

name	dept	age	salary	join_date
Sokha	Sales	28	1200	2021-03-15
Dara	IT	34	2100	2019-07-01
Mealea	Sales	25	1500	2022-01-20
Vibol	IT	41	2600	2018-11-05
Chan	HR	38	1800	2020-06-30

In [2]:
```
df.loc["Dara":"Vibol"]
```

Out[2]:

name	dept	age	salary	join_date
Dara	IT	34	2100	2019-07-01
Mealea	Sales	25	1500	2022-01-20
Vibol	IT	41	2600	2018-11-05

In [3]:
```
df.loc[["Dara", "Chan"], ["dept", "salary"]]
```

Out[3]:

name	dept	salary
Dara	IT	2100
Chan	HR	1800

Boolean Filtering

Keep rows that match

- A comparison like df["salary"] > 1500 returns a Series of True/False.
- Put that mask in df[...] and Pandas keeps only the True rows.
- Naming the mask first often reads more clearly.

In [1]:
```
import pandas as pd

df = pd.DataFrame({"name":   ["Alice", "Bob", "Carol", "David", "Eva"],
                   "dept":   ["Sales", "IT", "HR", "IT", "Sales"],
                   "salary": [1400, 2200, 1800, 1300, 2600]})

df
```

Out[1]:

	name	dept	salary
0	Alice	Sales	1400
1	Bob	IT	2200
2	Carol	HR	1800
3	David	IT	1300
4	Eva	Sales	2600

In [2]:
```
mask = df["salary"] > 1500      # a True/False Series
mask
```

Out[2]:
```
0    False
1     True
2     True
3    False
4     True
Name: salary, dtype: bool
```

In [3]:
```
df[mask]                        # keep only the True rows
```

Out[3]:

	name	dept	salary
1	Bob	IT	2200
2	Carol	HR	1800
4	Eva	Sales	2600

Combine Conditions

MULTIPLE CONDITIONS

Join masks with & | ~.

- **&** = and, **|** = or, **~** = not — bitwise operators, not the words.
- Wrap each condition in **parentheses**.
- **isin()** checks membership; **between()** checks an inclusive range.

Insight: Forgetting the **parentheses** is the #1 filtering bug — Python's operator precedence breaks the mask.

```
In [1]:
import pandas as pd

df = pd.DataFrame({"name":   ["Alice", "Bob", "Carol", "David", "Eva"],
                   "dept":   ["Sales", "IT", "HR", "IT", "Sales"],
                   "age":    [28, 41, 35, 23, 52],
                   "salary": [1400, 2200, 1800, 1300, 2600]})

df
```

Out[1]:

	name	dept	age	salary
0	Alice	Sales	28	1400
1	Bob	IT	41	2200
2	Carol	HR	35	1800
3	David	IT	23	1300
4	Eva	Sales	52	2600

```
In [2]: df[(df["dept"] == "Sales") & (df["age"] < 30)]["name"].tolist()   # AND
Out[2]: ['Alice']

In [3]: df[(df["dept"] == "HR") | (df["salary"] > 2000)]["name"].tolist() # OR
Out[3]: ['Bob', 'Carol', 'Eva']

In [4]: df[~(df["dept"] == "IT")]["name"].tolist()                        # NOT
Out[4]: ['Alice', 'Carol', 'Eva']

In [5]: df[df["dept"].isin(["Sales", "HR"])]["name"].tolist()   # membership
Out[5]: ['Alice', 'Carol', 'Eva']

In [6]: df[df["age"].between(25, 35)]["name"].tolist()          # inclusive range
Out[6]: ['Alice', 'Carol']
```

Transforming Columns

Turn existing columns into new ones with your own logic.

Explore a Column

First moves on any new column

- value_counts() tallies how often each value appears.
- unique() lists the distinct values.
- nunique() counts how many distinct values there are.

In [1]:
```
import pandas as pd

df = pd.DataFrame({"name":   ["Alice", "Bob", "Carol", "David", "Eva"],
                   "dept":   ["Sales", "IT", "HR", "IT", "Sales"],
                   "salary": [1400, 2200, 1800, 1300, 2600]})

df
```

Out[1]:

	name	dept	salary
0	Alice	Sales	1400
1	Bob	IT	2200
2	Carol	HR	1800
3	David	IT	1300
4	Eva	Sales	2600

In [2]:
```
df["dept"].value_counts()   # how often each value appears
```

Out[2]:
```
dept
Sales    2
IT       2
HR       1
Name: count, dtype: int64
```

In [3]:
```
df["dept"].unique()         # distinct values
```

Out[3]:
```
array(['Sales', 'IT', 'HR'], dtype=object)
```

In [4]:
```
df["dept"].nunique()        # how many distinct
```

Out[4]:
```
3
```

Create / Update new column

```
numbers = np.arange(1, 21).reshape(5,4)
df = pd.DataFrame(numbers, columns=["c1", "c2", "c3", "c4"])
df
```

	c1	c2	c3	c4
0	1	2	3	4
1	5	6	7	8
2	9	10	11	12
3	13	14	15	16
4	17	18	19	20

```
df["row_n"] = ["r1", "r2", "r3", 'r4', 'r5' ]
df
```

	c1	c2	c3	c4	row_n
0	1	2	3	4	r1
1	5	6	7	8	r2
2	9	10	11	12	r3
3	13	14	15	16	r4
4	17	18	19	20	r5

Create / Update new column by other column

```
df["total"] = df["c1"] + df["c2"] + df["c3"]+ df["c4"]
df
```

	c1	c2	c3	c4	row_n	total
0	1	2	3	4	r1	10
1	5	6	7	8	r2	26
2	9	10	11	12	r3	42
3	13	14	15	16	r4	58
4	17	18	19	20	r5	74

Sort Values

Order the rows

- sort_values() orders rows by one column.
- ascending=False puts the biggest first.
- Pass a list to sort by several columns — ties fall to the next key.

In [1]:
```
import pandas as pd

df = pd.DataFrame({"name":   ["Alice", "Bob", "Carol", "David", "Eva"],
                   "dept":   ["Sales", "IT", "HR", "IT", "Sales"],
                   "salary": [1400, 2200, 1800, 1300, 2600]})

df
```

Out[1]:

	name	dept	salary
0	Alice	Sales	1400
1	Bob	IT	2200
2	Carol	HR	1800
3	David	IT	1300
4	Eva	Sales	2600

In [2]:
```
df.sort_values("salary", ascending=False)
```

Out[2]:

	name	dept	salary
4	Eva	Sales	2600
1	Bob	IT	2200
2	Carol	HR	1800
0	Alice	Sales	1400
3	David	IT	1300

In [3]:
```
df.sort_values(["dept", "salary"])     # several columns
```

Out[3]:

	name	dept	salary
2	Carol	HR	1800
3	David	IT	1300
1	Bob	IT	2200
0	Alice	Sales	1400
4	Eva	Sales	2600

Apply & lambda

When built-ins aren't enough.

- apply() runs a function on every value of a Series.
- A lambda is a tiny inline function — great for a quick rule.
- On a DataFrame, axis=1 applies the function row by row.

In [1]:
```
import pandas as pd

df = pd.DataFrame({"name":   ["Alice", "Bob", "Carol", "David", "Eva"],
                   "age":    [28, 41, 35, 23, 52],
                   "salary": [1400, 2200, 1800, 1300, 2600]})

df
```

Out[1]:

	name	age	salary
0	Alice	28	1400
1	Bob	41	2200
2	Carol	35	1800
3	David	23	1300
4	Eva	52	2600

In [2]:
```
# Series.apply: one value at a time
df["level"] = df["age"].apply(lambda x: "senior" if x >= 35 else "junior")

# axis=1: one whole row at a time
df["ratio"] = df.apply(lambda r: round(r["salary"] / r["age"], 1), axis=1)
df
```

Out[2]:

	name	age	salary	level	ratio
0	Alice	28	1400	junior	50.0
1	Bob	41	2200	senior	53.7
2	Carol	35	1800	senior	51.4
3	David	23	1300	junior	56.5
4	Eva	52	2600	senior	50.0

Swap values using a lookup (dict).

Using map with a dict

- map() replaces each value using a dict of old to new.
- Perfect for turning categories into short codes or labels.
- Any value missing from the dict becomes NaN.

- **Insight:** use map for value-to-value swaps; reach for apply only when you need real logic.

In [1]:
```
import pandas as pd

df = pd.DataFrame({"name": ["Alice", "Bob", "Carol", "David", "Eva"],
                   "dept": ["Sales", "IT", "HR", "IT", "Sales"]})

df
```

Out[1]:

	name	dept
0	Alice	Sales
1	Bob	IT
2	Carol	HR
3	David	IT
4	Eva	Sales

In [2]:
```
codes = {"Sales": "S", "IT": "T", "HR": "H"}
df["dept_code"] = df["dept"].map(codes)
df
```

Out[2]:

	name	dept	dept_code
0	Alice	Sales	S
1	Bob	IT	T
2	Carol	HR	H
3	David	IT	T
4	Eva	Sales	S

In [3]:
```
df["dept"].map({"Sales": "S"}).tolist()   # missing keys -> NaN
```

Out[3]:
```
['S', nan, nan, nan, 'S']
```

String Methods

The .str accessor

- Prefix a text Series with .str to use string methods on every row.
- lower(), len(), startswith(), replace() all work element-wise.
- contains() returns a mask — so it plugs straight into filtering.

```
In [1]: import pandas as pd

        df = pd.DataFrame({"name": ["Alice", "bob", "Carol", "DAVID", "eva"]})

        df
```

Out[1]:

	name
0	Alice
1	bob
2	Carol
3	DAVID
4	eva

```
In [2]: df["name"].str.lower().tolist()
Out[2]: ['alice', 'bob', 'carol', 'david', 'eva']
In [3]: df["name"].str.len().tolist()
Out[3]: [5, 3, 5, 5, 3]
In [4]: df["name"].str.startswith("A").tolist()
Out[4]: [True, False, False, False, False]
In [5]: df["name"].str.replace("a", "@").tolist()
Out[5]: ['Alice', 'bob', 'C@rol', 'DAVID', 'ev@']
In [6]: df[df["name"].str.contains("a")]   # mask -> plugs into filtering
```

Out[6]:

	name
2	Carol
4	eva

Pivot & melt

Wide and long

- pivot_table: one summary number per group.
- melt: many columns collapse into rows.

In [1]:
```
import pandas as pd

df = pd.DataFrame({"name":   ["Alice", "Bob", "Carol", "David"],
                   "dept":   ["Sales", "IT", "Sales", "IT"],
                   "age":    [28, 41, 35, 23],
                   "salary": [1400, 2200, 1800, 1300]})

df
```

Out[1]:

	name	dept	age	salary
0	Alice	Sales	28	1400
1	Bob	IT	41	2200
2	Carol	Sales	35	1800
3	David	IT	23	1300

In [2]:
```
df.pivot_table(index="dept", values="salary", aggfunc="mean")   # long -> wide
```

Out[2]:

	salary
dept	
IT	1750.0
Sales	1600.0

In [3]:
```
pd.melt(df.head(3), id_vars=["name"], value_vars=["age", "salary"])   # wide -> long
```

Out[3]:

	name	variable	value
0	Alice	age	28
1	Bob	age	41
2	Carol	age	35
3	Alice	salary	1400
4	Bob	salary	2200
5	Carol	salary	1800

Parse Dates

Text into real dates

- Strings can't do date math.
- pd.to_datetime() converts a text column into real datetimes.
- Then the .dt accessor unlocks year, month, day_name(), and more.
- Do this once, up front — everything downstream gets easier.

In [1]:
```
import pandas as pd

df = pd.DataFrame({"name": ["Alice", "Bob", "Carol"],
                   "join_date": ["2021-03-15", "2019-07-01", "2022-11-30"]})

df
```

Out[1]:

	name	join_date
0	Alice	2021-03-15
1	Bob	2019-07-01
2	Carol	2022-11-30

In [2]:
```
df.dtypes                          # text, not dates
```

Out[2]:
```
name          object
join_date     object
dtype: object
```

In [3]:
```
df["join_date"] = pd.to_datetime(df["join_date"])
df.dtypes                          # real datetimes now
```

Out[3]:
```
name                  object
join_date     datetime64[ns]
dtype: object
```

In [4]:
```
df["year"] = df["join_date"].dt.year
df["month"] = df["join_date"].dt.month
df["weekday"] = df["join_date"].dt.day_name()
df
```

Out[4]:

	name	join_date	year	month	weekday
0	Alice	2021-03-15	2021	3	Monday
1	Bob	2019-07-01	2019	7	Monday
2	Carol	2022-11-30	2022	11	Wednesday

Date Math & Resample

Math with time

- Subtracting two datetimes gives a duration; .dt.days turns it into a number.
- resample() groups a datetime-indexed series into time buckets.
- "YE" = year-end, "ME" = month-end, "W" = weekly.

- **Insight:** resample only works when the index is a datetime — that's what set_index is for here.

In [1]:
```
import pandas as pd

df = pd.DataFrame({"name": ["Alice", "Bob", "Carol", "David", "Eva"],
                   "join_date": pd.to_datetime(["2021-03-15", "2019-07-01",
                       "2021-11-30", "2022-05-10", "2019-01-20"]),
                   "salary": [1400, 2200, 1800, 1300, 2600]})

df
```

Out[1]:

	name	join_date	salary
0	Alice	2021-03-15	1400
1	Bob	2019-07-01	2200
2	Carol	2021-11-30	1800
3	David	2022-05-10	1300
4	Eva	2019-01-20	2600

In [2]:
```
# datetime - datetime = duration; .dt.days makes it a number
df["tenure_days"] = (pd.Timestamp("2026-01-01") - df["join_date"]).dt.days
df[["name", "tenure_days"]]
```

Out[2]:

	name	tenure_days
0	Alice	1753
1	Bob	2376
2	Carol	1493
3	David	1332
4	Eva	2538

In [3]:
```
# resample needs a datetime index
ts = df.set_index("join_date")["salary"]
ts.resample("YE").mean()       # average salary per year
```

Out[3]:
```
join_date
2019-12-31    2400.0
2020-12-31       NaN
2021-12-31    1600.0
2022-12-31    1300.0
Freq: YE-DEC, Name: salary, dtype: float64
```

Missing Value

Detect Missing

NaN awareness

- .isna() returns a boolean mask of missing cells.
- .isna().sum() counts gaps per column.
- .notna() is the inverse.
- Spot problems before you analyze.

In [1]:
```
import pandas as pd

df = pd.DataFrame({"c1": [1, 2, None], "c2": [4, 5, 6], "c3": [7, None, 9]})

df
```

Out[1]:

	c1	c2	c3
0	1.0	4	7.0
1	2.0	5	NaN
2	NaN	6	9.0

In [2]:
```
df.isna()                # True where a value is missing
```

Out[2]:

	c1	c2	c3
0	False	False	False
1	False	False	True
2	True	False	False

In [3]:
```
df.isna().sum()          # count NaN per column
```

Out[3]:
```
c1    1
c2    0
c3    1
dtype: int64
```

In [4]:
```
df.notna().sum()         # inverse: count valid values
```

Out[4]:
```
c1    2
c2    3
c3    2
dtype: int64
```

In [5]:
```
df.isna().values.any()   # any missing anywhere?
```

Out[5]:
```
np.True_
```

Drop Missing

dropna by row or column

- .dropna() removes any row with a NaN.
- axis=1 drops offending columns instead.
- how='all' only drops fully-empty rows.
- thresh=n keeps rows with n non-null values.

```
In [1]: import pandas as pd

        df = pd.DataFrame({"c1": [1, 2, None], "c2": [4, 5, 6], "c3": [7, None, 9]})

        df
```

Out[1]:

	c1	c2	c3
0	1.0	4	7.0
1	2.0	5	NaN
2	NaN	6	9.0

```
In [2]: df.dropna()              # drop ROWS with any NaN
```

Out[2]:

	c1	c2	c3
0	1.0	4	7.0

```
In [3]: df.dropna(axis=1)        # drop COLUMNS with any NaN
```

Out[3]:

	c2
0	4
1	5
2	6

```
In [4]: df.dropna(how="all")     # only fully-empty rows
```

Out[4]:

	c1	c2	c3
0	1.0	4	7.0
1	2.0	5	NaN
2	NaN	6	9.0

```
In [5]: df.dropna(thresh=3)      # keep rows with >= 3 values
```

Out[5]:

	c1	c2	c3
0	1.0	4	7.0

Fill Missing

fillna with a value

- .fillna(0) replaces gaps with a constant.
- We may fill a column with its own mean or median to keeps the row count intact.
- Choose the fill that fits the data's meaning.

Insight: the mean is common, but the median is safer when a column has outliers.

In [1]:
```
import pandas as pd

df = pd.DataFrame({"c1": [1, 2, None], "c2": [4, 5, 6], "c3": [7, None, 9]})

df
```

Out[1]:

	c1	c2	c3
0	1.0	4	7.0
1	2.0	5	NaN
2	NaN	6	9.0

In [2]:
```
df.fillna(0)                # a constant
```

Out[2]:

	name	age	salary
0	Alice	28.0	1400.0
1	Bob	0.0	2200.0
2	Carol	35.0	0.0
3	David	23.0	1300.0

In [3]:
```
df["age"] = df["age"].fillna(df["age"].mean())
df["salary"] = df["salary"].fillna(df["salary"].median())
df
```

Out[3]:

	name	age	salary
0	Alice	28.000000	1400.0
1	Bob	28.666667	2200.0
2	Carol	35.000000	1400.0
3	David	23.000000	1300.0

Fill by Aggregation

Use the column's own statistic

- Compute the statistic first, then hand it to fillna().
- Assign the result back with df[col] = ...
- Each column can use a different rule.

In [1]:
```
import pandas as pd

df = pd.DataFrame({"c1": [1, 2, None], "c2": [4, 5, 6], "c3": [7, None, 9]})

df
```

Out[1]:

	c1	c2	c3
0	1.0	4	7.0
1	2.0	5	NaN
2	NaN	6	9.0

In [2]:
```
# fill c1 with its own mean, c3 with its median
df["c1"] = df["c1"].fillna(df["c1"].mean())
df["c3"] = df["c3"].fillna(df["c3"].median())
df
```

Out[2]:

	c1	c2	c3
0	1.0	4	7.0
1	2.0	5	8.0
2	1.5	6	9.0

Aggregation

Aggregate a DataFrame

By row or column

- axis=0 aggregates down columns (the default).
- axis=1 aggregates across rows.
- .agg([...]) runs several functions at once.

In [1]:
```
import pandas as pd

df = pd.DataFrame({"c1": [1, 2, 3, 4], "c2": [2, 4, 6, 8],
                   "c3": [3, 6, 9, 12], "c4": [4, 8, 12, 16]})

df
```

Out[1]:

	c1	c2	c3	c4
0	1	2	3	4
1	2	4	6	8
2	3	6	9	12
3	4	8	12	16

In [2]:
```
df.sum(axis=0)       # down columns (default)
```

Out[2]:
```
c1    10
c2    20
c3    30
c4    40
dtype: int64
```

In [3]:
```
df.sum(axis=1)       # across rows
```

Out[3]:
```
0    10
1    20
2    30
3    40
dtype: int64
```

In [4]:
```
df.agg(["min", "max", "mean"])
```

Out[4]:

	c1	c2	c3	c4
min	1.0	2.0	3.0	4.0
max	4.0	8.0	12.0	16.0
mean	2.5	5.0	7.5	10.0

Group By

The groupby method is to group rows of data together and call aggregate functions

groupby()

Split, apply, combine

- df.groupby('col') splits rows into groups.
- Chain an aggregate: .mean() / .sum() / .size().
- The grouping column becomes the new index.
- One line replaces a manual loop over categories.

In [1]:
```
import pandas as pd

df = pd.DataFrame({
    "city":  ["Phnom Penh", "Siem Reap", "Phnom Penh",
              "Siem Reap", "Siem Reap", "Phnom Penh"],
    "age":   [28, 41, 35, 23, 52, 30],
    "score": [80, 92, 75, 88, 60, 95],
})

df
```

Out[1]:

	city	age	score
0	Phnom Penh	28	80
1	Siem Reap	41	92
2	Phnom Penh	35	75
3	Siem Reap	23	88
4	Siem Reap	52	60
5	Phnom Penh	30	95

In [2]:
```
df.groupby("city")["age"].mean()
```

Out[2]:
```
city
Phnom Penh    31.000000
Siem Reap     38.666667
Name: age, dtype: float64
```

In [3]:
```
df.groupby("city")[["age", "score"]].sum()
```

Out[3]:

city	age	score
Phnom Penh	93	250
Siem Reap	116	240

In [4]:
```
df.groupby("city").size()      # rows per group
```

Out[4]:
```
city
Phnom Penh    3
Siem Reap     3
dtype: int64
```

Describe by Group

Richer summaries

- .describe() gives full stats per group.
- .agg({...}) applies different functions per column.
- Combine several metrics in one call.
- Great for comparing categories side by side.

In [1]:
```
import pandas as pd

df = pd.DataFrame({"Company": ["GOOG", "GOOG", "MSFT", "MSFT", "FB", "FB"],
                   "Sales":   [200, 120, 340, 124, 243, 350]})

df
```

Out[1]:

	Company	Sales
0	GOOG	200
1	GOOG	120
2	MSFT	340
3	MSFT	124
4	FB	243
5	FB	350

In [2]:
```
df.groupby("Company")["Sales"].describe().round(1)   # full stats
```

Out[2]:

Company	count	mean	std	min	25%	50%	75%	max
FB	2.0	296.5	75.7	243.0	269.8	296.5	323.2	350.0
GOOG	2.0	160.0	56.6	120.0	140.0	160.0	180.0	200.0
MSFT	2.0	232.0	152.7	124.0	178.0	232.0	286.0	340.0

In [3]:
```
df.groupby("Company").agg({"Sales": ["mean", "max", "sum"]})
```

Out[3]:

	Sales		
	mean	max	sum
Company			
FB	296.5	350	593
GOOG	160.0	200	320
MSFT	232.0	340	464

Combine multiple data into one

Three Ways to Combine

OVERVIEW

1

Concatenate

Glue frames together along an axis — stack rows or columns. Dimensions must match.

2

Merge

Combine on a key column, exactly like a SQL join.

3

Join

Combine on the index of two differently-indexed frames.

Concatenate

Concatenation basically glues together DataFrames. Keep in mind that dimensions should match along the axis you are concatenating on. You can use pd.concat and pass in a list of DataFrames to concatenate together:

```
students = pd.DataFrame({
        "id": [1,2,3],
        "name": ["Jame", "John", "Jane"],
        "gender": ["M", "M", "F"],
    },
    index=[1,2,3]
)
students
```

	id	name	gender
1	1	Jame	M
2	2	John	M
3	3	Jane	F

```
students2 = pd.DataFrame({
        "id": [4,5,6],
        "name": ["Ben", "Brown", "Bill"],
    },
    index=[3,4,5]
)
students2
```

	id	name
3	4	Ben
4	5	Brown
5	6	Bill

Concatenate Rows

Glue frames together

- pd.concat([...]) stacks a list of frames.
- Vertical (axis=0) adds rows — columns that don't match become NaN.
- ignore_index=True renumbers the rows after stacking.

In [1]:
```
import pandas as pd

students = pd.DataFrame(
    {"id": [1, 2, 3], "name": ["Jame", "John", "Jane"],
     "gender": ["M", "M", "F"]}, index=[1, 2, 3])
students
```

Out[1]:

	id	name	gender
1	1	Jame	M
2	2	John	M
3	3	Jane	F

In [2]:
```
students2 = pd.DataFrame(
    {"id": [4, 5, 6], "name": ["Ben", "Brown", "Bill"]},
    index=[3, 4, 5])
students2
```

Out[2]:

	id	name
3	4	Ben
4	5	Brown
5	6	Bill

In [3]:
```
pd.concat([students, students2])                    # stack rows
```

Out[3]:

	id	name	gender
1	1	Jame	M
2	2	John	M
3	3	Jane	F
3	4	Ben	NaN
4	5	Brown	NaN
5	6	Bill	NaN

In [4]:
```
pd.concat([students, students2], ignore_index=True)    # renumber index
```

Out[4]:

	id	name	gender
0	1	Jame	M
1	2	John	M
2	3	Jane	F
3	4	Ben	NaN
4	5	Brown	NaN
5	6	Bill	NaN

Concatenate Columns

Side by side

- Horizontal (axis=1) adds columns, aligned on the index.
- Rows with no partner get NaN.
- Check your index first — misaligned indexes make gaps.

In [1]:
```
import pandas as pd

students = pd.DataFrame(
    {"id": [1, 2, 3], "name": ["Jame", "John", "Jane"],
     "gender": ["M", "M", "F"]}, index=[1, 2, 3])
students
```

Out[1]:

	id	name	gender
1	1	Jame	M
2	2	John	M
3	3	Jane	F

In [2]:
```
students2 = pd.DataFrame(
    {"id": [4, 5, 6], "name": ["Ben", "Brown", "Bill"]},
    index=[3, 4, 5])
students2
```

Out[2]:

	id	name
3	4	Ben
4	5	Brown
5	6	Bill

In [3]:
```
pd.concat([students, students2], axis=1)   # side by side, aligned on index
```

Out[3]:

	id	name	gender	id	name
1	1.0	Jame	M	NaN	NaN
2	2.0	John	M	NaN	NaN
3	3.0	Jane	F	4.0	Ben
4	NaN	NaN	NaN	5.0	Brown
5	NaN	NaN	NaN	6.0	Bill

Merge

The merge function allows you to merge DataFrames together using a similar logic as merging SQL Tables together. For example

```
students = pd.DataFrame({
        "id": [1,2,3],
        "name": ["Jame", "John", "Jane"],
        "gender": ["M", "M", "F"],
        "class_id": [1,1,2]
    },
    index=[1,2,3]
)
students
```

	id	name	gender	class_id
1	1	Jame	M	1
2	2	John	M	1
3	3	Jane	F	2

```
classes = pd.DataFrame({
        "id": [1,2,3],
        "name": ["Data Science", "AI", "Machine Learning"]
    },
    index=[1,2,3]
)
classes
```

	id	name
1	1	Data Science
2	2	AI
3	3	Machine Learning

```
students.merge(classes, left_on="class_id", right_on="id" )
```

	id_x	name_x	gender	class_id	id_y	name_y
0	1	Jame	M	1	1	Data Science
1	2	John	M	1	1	Data Science
2	3	Jane	F	2	2	AI

Merge

SQL-style on a key

- pd.merge(l, r, on='id') joins on a shared column.
- how= picks the join type: inner, left, right, outer.
- left_on / right_on when key names differ.
- Same-named columns get _x / _y suffixes.

In [1]:
```
import pandas as pd

students = pd.DataFrame({"id": [1, 2, 3], "name": ["Jame", "John", "Jane"],
                         "class_id": [1, 1, 2]})
students
```

Out[1]:

	id	name	class_id
0	1	Jame	1
1	2	John	1
2	3	Jane	2

In [2]:
```
classes = pd.DataFrame({"id": [1, 2, 3],
                        "name": ["Data Science", "AI", "Machine Learning"]})
classes
```

Out[2]:

	id	name
0	1	Data Science
1	2	AI
2	3	Machine Learning

In [3]:
```
# key names differ -> left_on / right_on
students.merge(classes, left_on="class_id", right_on="id")
```

Out[3]:

	id_x	name_x	class_id	id_y	name_y
0	1	Jame	1	1	Data Science
1	2	John	1	1	Data Science
2	3	Jane	2	2	AI

In [4]:
```
# how= picks the join type (class 3 has no students)
students.merge(classes, how="right", left_on="class_id", right_on="id")
```

Out[4]:

	id_x	name_x	class_id	id_y	name_y
0	1.0	Jame	1.0	1	Data Science
1	2.0	John	1.0	1	Data Science
2	3.0	Jane	2.0	2	AI
3	NaN	NaN	NaN	3	Machine Learning

Join

Combine on the index

- left.join(right) aligns on the row index.
- Convenient when frames are indexed the same way.
- how= controls which keys survive.
- Under the hood it's merge on the index.

join.py

```
# Join on the INDEX of both frames
left.join(right)

left.join(right, how='left')
left.join(right, how='outer')
```

Join

Joining is a convenient method for combining the columns of two potentially differently-indexed DataFrames into a single result DataFrame.

join.py

```
# Join on the INDEX of both frames
left.join(right)

left.join(right, how='left')
left.join(right, how='outer')
```

```
students = pd.DataFrame({
        "id": [1,2,3],
        "name": ["Jame", "John", "Jane"],
    },
    index=[1,2,3]
)
students
```

	id	name
1	1	Jame
2	2	John
3	3	Jane

```
students2 = pd.DataFrame({
        "DOB": ["2000-01-24" , "1998-12-01", "2000-02-02"],
        "gender": ["M", "M", "F"],
    },
    index=[3,4,5]
)
students2
```

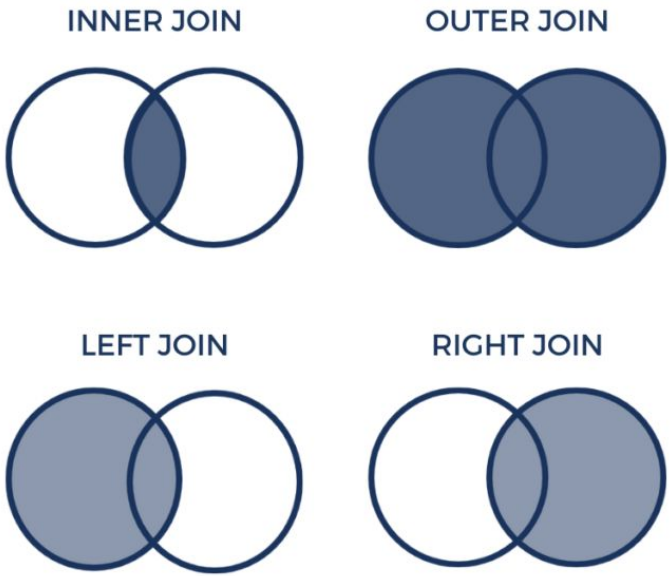
	DOB	gender
3	2000-01-24	M
4	1998-12-01	M
5	2000-02-02	F

```
students.join(students2)
```

	id	name	DOB	gender
1	1	Jame	NaN	NaN
2	2	John	NaN	NaN
3	3	Jane	2000-01-24	M

Strategy for Join / Merge

Strategy for Join / Merge



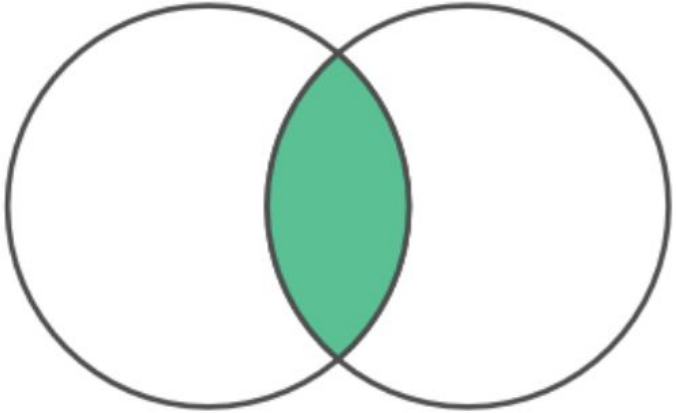
Inner

```
students = pd.DataFrame({
        "id": [1,2,3],
        "name": ["Jame", "John", "Jane"],
    },
    index=[1,2,3]
)
students
```

	id	name
1	1	Jame
2	2	John
3	3	Jane

```
students2 = pd.DataFrame({
        "DOB": ["2000-01-24" , "1998-12-01", "2000-02-02"],
        "gender": ["M", "M", "F"],
    },
    index=[3,4,5]
)
students2
```

	DOB	gender
3	2000-01-24	M
4	1998-12-01	M
5	2000-02-02	F




```
students.join(students2, how='inner')
```

	id	name	DOB	gender
3	3	Jane	2000-01-24	M

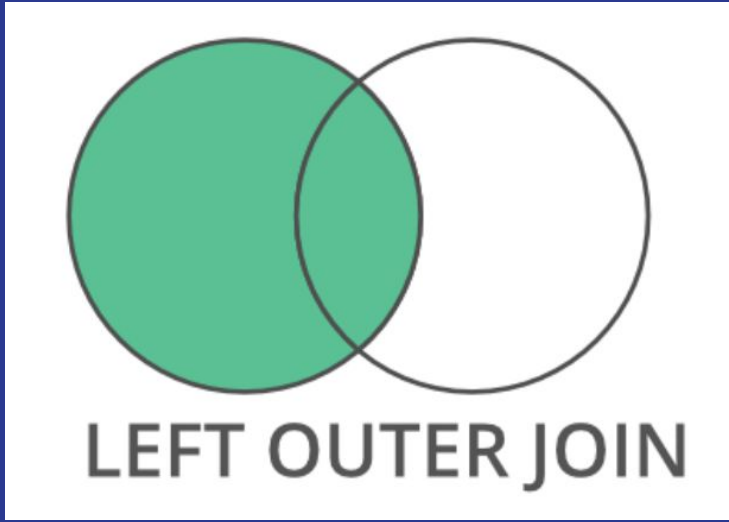
Left (Outer) Join

```
students = pd.DataFrame({
        "id": [1,2,3],
        "name": ["Jame", "John", "Jane"],
    },
    index=[1,2,3]
)
students
```

	id	name
1	1	Jame
2	2	John
3	3	Jane

```
students2 = pd.DataFrame({
        "DOB": ["2000-01-24" , "1998-12-01", "2000-02-02"],
        "gender": ["M", "M", "F"],
    },
    index=[3,4,5]
)
students2
```

	DOB	gender
3	2000-01-24	M
4	1998-12-01	M
5	2000-02-02	F




```
students.join(students2, how='left')
```

	id	name	DOB	gender
1	1	Jame	NaN	NaN
2	2	John	NaN	NaN
3	3	Jane	2000-01-24	M

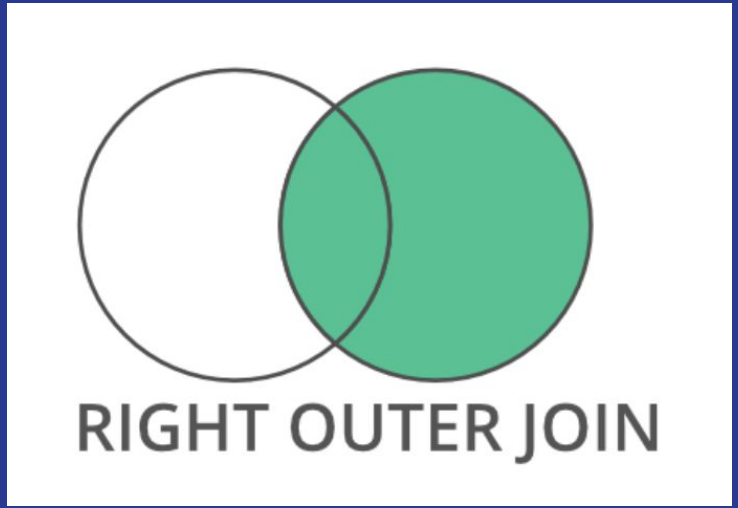
Right (Outer) Join

```
students = pd.DataFrame({
        "id": [1,2,3],
        "name": ["Jame", "John", "Jane"],
    },
    index=[1,2,3]
)
students
```

	id	name
1	1	Jame
2	2	John
3	3	Jane

```
students2 = pd.DataFrame({
        "DOB": ["2000-01-24" , "1998-12-01", "2000-02-02"],
        "gender": ["M", "M", "F"],
    },
    index=[3,4,5]
)
students2
```

	DOB	gender
3	2000-01-24	M
4	1998-12-01	M
5	2000-02-02	F




```
students.join(students2, how='right')
```

✓ 0.0s

	id	name	DOB	gender
3	3.0	Jane	2000-01-24	M
4	NaN	NaN	1998-12-01	M
5	NaN	NaN	2000-02-02	F

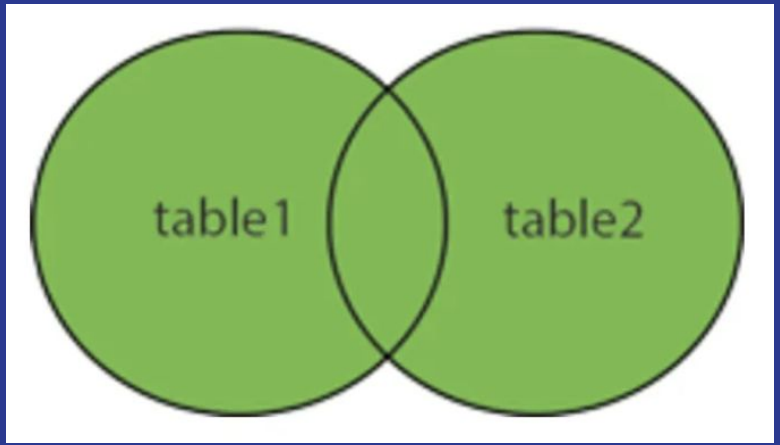
Outer Join

```
students = pd.DataFrame({
        "id": [1,2,3],
        "name": ["Jame", "John", "Jane"],
    },
    index=[1,2,3]
)
students
```

	id	name
1	1	Jame
2	2	John
3	3	Jane

```
students2 = pd.DataFrame({
        "DOB": ["2000-01-24" , "1998-12-01", "2000-02-02"],
        "gender": ["M", "M", "F"],
    },
    index=[3,4,5]
)
students2
```

	DOB	gender
3	2000-01-24	M
4	1998-12-01	M
5	2000-02-02	F




```
students.join(students2, how='outer')
```

	id	name	DOB	gender
1	1.0	Jame	NaN	NaN
2	2.0	John	NaN	NaN
3	3.0	Jane	2000-01-24	M
4	NaN	NaN	1998-12-01	M
5	NaN	NaN	2000-02-02	F

Pandas Data Practice Exercises

Pandas.DataFrame

LESSON SLIDES 2-5

Building a frame from a dict, a list of dicts or Series — then reading its shape, dtypes and index. (6 exercises)

Creating & Inspecting · Level 1

1 Shape and dtypes

- Show the shape of shop.
- Show the dtype of every column.

```
shop = pd.DataFrame({
    "item":  ["pen", "book", "bag"],
    "qty":   [3, 1, 2],
    "price": [1.5, 12.0, 25.0],
})
```

Expected output:

```
[1] (3, 3)

[2] item      object
    qty        int64
    price    float64
    dtype: object
```

2 Head, tail, columns

- Show the first two rows.
- Show the last row.
- Show the column names.

```
temps = pd.DataFrame({
    "city": ["PP", "Siem Reap",
             "Kampot", "Kep"],
    "high": [34, 33, 31, 30],
})
```

Expected output:

[1]

	city	high
0	PP	34
1	Siem Reap	33

[2]

	city	high
3	Kep	30

```
[3] Index(['city', 'high'], dtype='object')
```

Read / Query DataFrame

LESSON SLIDES 6-12

Getting a column, loc vs iloc, slicing by position and by label, boolean filtering and combining conditions. (12 exercises)

Selecting Columns & Rows · Level 1

1 Series or DataFrame?

- Select the fruit column with single brackets([ ]).
- Select the fruit column with double brackets([[ ]]) .
- Select the fruit and kg columns.

```
basket = pd.DataFrame({
    "fruit": ["mango", "banana", "apple"],
    "kg":    [2, 5, 1],
})
```

Expected output:

[1]
```
0     mango
1    banana
2     apple
Name: fruit, dtype: object
```

[2]

	fruit
0	mango
1	banana
2	apple

[3]

	fruit	kg
0	mango	2
1	banana	5
2	apple	1

Selecting Columns & Rows

2 When the index is text

- Select row B by label.
- Select the row at position 1.
- Select row 1 by label. What happens?

```
ref = pd.DataFrame(
    {"val": [10, 20, 30]},
    index=["A", "B", "C"],
)
```

Expected output:

```
[1] val    20
    Name: B, dtype: int64

[2] val    20
    Name: B, dtype: int64

[3] KeyError: 1
```

3 Lists of labels

- Select the dept and salary of Dara and Chan.
- Select every row from Dara to Vibol.
- Select Dara and Zed. What happens?

```
staff = pd.DataFrame({
    "dept":   ["IT", "HR", "IT", "Ops"],
    "salary": [900, 700, 1200, 800],
}, index=["Dara", "Chan", "Vibol", "Sok"])
```

Expected output:

[1]

	dept	salary
Dara	IT	900
Chan	HR	700

[2]

	dept	salary
Dara	IT	900
Chan	HR	700
Vibol	IT	1200

```
[3] KeyError: "['Zed'] not in index"
```

Filtering Rows

4 A simple mask

- Select the students who scored 60 or more.
- Count how many students passed.

```
cls = pd.DataFrame({
    "name":  ["Sok", "Dara", "Mony", "Vann"],
    "score": [45, 72, 88, 59],
})
```

Expected output:

[1]

	name	score
1	Dara	72
2	Mony	88

[2] 2

5 Name the mask

- Build a mask for red cars and show it.
- Use the mask to select the red cars.

```
cars = pd.DataFrame({
    "colour": ["red", "blue", "red"],
    "km":     [100, 250, 80],
})
```

Expected output:

[1]
```
0     True
1    False
2     True
Name: colour, dtype: bool
```

[2]

	colour	km
0	red	100
2	red	80

Filtering Rows

6 AND, OR, NOT

- Select orders in PP and with qty above 2.
- Select orders in PP or with qty above 4.
- Select the orders not in PP.

```
o = pd.DataFrame({
    "city": ["PP", "SR", "PP", "BB"],
    "qty":  [3, 5, 1, 2],
})
```

Expected output:

[1]

	city	qty
0	PP	3

[2]

	city	qty
0	PP	3
1	SR	5
2	PP	1

[3]

	city	qty
1	SR	5
3	BB	2

Transforming Columns

Exploring & Sorting

1 Tally a column

- Count each colour.
- Count the rows.

```
v = pd.DataFrame({
    "colour": ["red", "blue", "red",
               "red", "blue"],
})
```

Expected output:

```
[1] colour
    red     3
    blue    2
    Name: count, dtype: int64

[2] 5
```

2 Distinct values

- List the distinct sizes.
- Count the distinct sizes.
- Count the rows.

```
s = pd.DataFrame({
    "size": ["S", "M", "S", "L", "M"],
})
```

Expected output:

```
[1] array(['S', 'M', 'L'], dtype=object)

[2] 3

[3] 5
```

Exploring & Sorting

3 Sort by one key, then two

- Sort the results by time.
- Sort by team, then by time.

```
r = pd.DataFrame({
    "team": ["B", "A", "B", "A"],
    "time": [12.5, 11.0, 10.5, 13.0],
})
```

Expected output:

[1]

	team	time
2	B	10.5
1	A	11.0
0	B	12.5
3	A	13.0

[2]

	team	time
1	A	11.0
3	A	13.0
2	B	10.5
0	B	12.5

Creating Columns

4 Multiply two columns

- Add an area column (w × h).
- Show the dtype of area.

```
r = pd.DataFrame({
    "w": [2, 3, 4],
    "h": [5, 5, 2],
})
```

Expected output:

[1]

	w	h	area
0	2	5	10
1	3	5	15
2	4	2	8

[2]
```
int64
```

5 A flat column, then overwrite

- Add a vat column of 0.25 to every row.
- Replace amt with the amount including VAT.

```
inv = pd.DataFrame({
    "line": [1, 2],
    "amt":  [100.0, 250.0],
})
```

Expected output:

[1]

	line	amt	vat
0	1	100.0	0.25
1	2	250.0	0.25

[2]

	line	amt	vat
0	1	125.0	0.25
1	2	312.5	0.25

Creating Columns

6 A column that is not there yet

- Add a big flag (total > 25) before total exists.
- Add total (qty × price) first, then the flag.

```
d = pd.DataFrame({
    "qty":   [2, 3],
    "price": [10.0, 20.0],
})
```

Expected output:

```
[1] KeyError: 'total'
```

[2]

	qty	price	total	big
0	2	10.0	20.0	False
1	3	20.0	60.0	True

7 Numbers stored as text

- Add qty to itself.
- Convert qty to numbers, then add again.

```
d = pd.DataFrame({
    "qty": ["2", "3"],
})
```

Expected output:

```
[1] 0    22
    1    33
    Name: qty, dtype: object

[2] 0    4
    1    6
    Name: qty, dtype: int64
```

apply, map & .str

8 apply with a lambda

- Add a size column: 'big' if n ≥ 3, else 'small'.
- Count each size.

```
d = pd.DataFrame({"n": [1, 3, 5, 2]})
```

Expected output:

[1]

	n	size
0	1	small
1	3	big
2	5	big
3	2	small

[2]

```
size
small    2
big      2
Name: count, dtype: int64
```

9 map with a dict

- Translate code with the words dict.
- Select the rows whose code has no translation.

```
d = pd.DataFrame({"code": ["S", "M", "X"]})
words = {"S": "small", "M": "medium"}
```

Expected output:

[1]

```
0     small
1    medium
2       NaN
Name: code, dtype: object
```

[2]

	code
2	X

Missing Value

Missing Values

1 Count the gaps

- Mark every missing cell.
- Count the gaps per column.
- Select the rows where a is missing.

```
d = pd.DataFrame({
    "a": [1.0, np.nan, 3.0],
    "b": [1.0, 2.0, 3.0],
})
```

Expected output:

[1]

	a	b
0	False	False
1	True	False
2	False	False

[2]

```
a    1
b    0
dtype: int64
```

[3]

	a	b
1	NaN	2.0

2 Show the affected rows

- Select the rows missing x.
- Select the rows missing y.
- Select the rows missing x or y.

```
d = pd.DataFrame({
    "id": [1, 2, 3],
    "x":  [5.0, np.nan, 7.0],
    "y":  [np.nan, 2.0, 3.0],
})
```

Expected output:

[1]

	id	x	y
1	2	NaN	2.0

[2]

	id	x	y
0	1	5.0	NaN

[3]

	id	x	y
0	1	5.0	NaN
1	2	NaN	2.0

Missing Values

3 Drop rows, then columns

- Drop the rows that have a gap.
- Drop the columns that have a gap.

```
d = pd.DataFrame({
    "a": [1.0, 2.0, np.nan],
    "b": [4.0, 5.0, 6.0],
})
```

Expected output:

[1]

	a	b
0	1.0	4.0
1	2.0	5.0

[2]

	b
0	4.0
1	5.0
2	6.0

Missing Values

4 Four fills, four answers

- Total qty × price after dropping the gap.
- Total it after filling price with 0.
- Total it after filling with the mean price.
- Total it after filling with the median price.

```
d = pd.DataFrame({
    "qty":   [2, 3, 1],
    "price": [10.0, np.nan, 20.0],
})
```

Expected output:

```
[1] 40.0
[2] 40.0
[3] 85.0
[4] 85.0
```

Aggregation

Aggregation · Level 1

1 Basic statistics

- Find the mean temperature.
- Find the median temperature.
- Find the lowest and the highest.

```
d = pd.DataFrame({
    "temp": [30, 34, 31, 35],
})
```

Expected output:

```
[1] 32.5
[2] 32.5
[3] 30 35
```

2 Counting with a mask

- Count the marks of 60 or more.
- Find the share of marks of 60 or more.

```
d = pd.DataFrame({
    "mark": [80, 45, 62, 59],
})
```

Expected output:

```
[1] 2
[2] 0.5
```

Aggregation · Level 3

3 Mean or median?

- Find the mean salary.
- Find the median salary.
- Find the mean without the 5000 row.

```
d = pd.DataFrame({
    "salary": [300, 320, 310, 5000],
})
```

Expected output:

```
[1] 1482.5
[2] 315.0
[3] 310.0
```

4 Mean of rates or rate of sums?

- Add a rate column (clicks ÷ views).
- Average the rate column.
- Divide total clicks by total views.

```
d = pd.DataFrame({
    "clicks": [1, 9],
    "views":  [4, 12],
})
```

Expected output:

[1]

	clicks	views	rate
0	1	4	0.25
1	9	12	0.75

```
[2] 0.5
[3] 0.625
```

Group By

Group By · Level 1

1 Average per group

- Average the sale per branch.
- Count the rows per branch.

```
d = pd.DataFrame({
    "branch": ["PP", "SR", "PP", "SR"],
    "sale":   [10.0, 4.0, 20.0, 6.0],
})
```

Expected output:

```
[1] branch
    PP    15.0
    SR     5.0
    Name: sale, dtype: float64

[2] branch
    PP    2
    SR    2
    dtype: int64
```

Group By · Level 1

2 Sum, max and the winner

- Total the sale per branch.
- Find the biggest single sale per branch.
- Sort the totals, highest first.

```
d = pd.DataFrame({
    "branch": ["PP", "SR", "PP"],
    "sale":   [10, 4, 20],
})
```

Expected output:

```
[1] branch
    PP    30
    SR     4
    Name: sale, dtype: int64

[2] branch
    PP    20
    SR     4
    Name: sale, dtype: int64

[3] branch
    PP    30
    SR     4
    Name: sale, dtype: int64
```

Combine multiple data into one

Combining Data · Level 1

1 Stack two frames

- Stack jan and feb.
- Stack them again, renumbering the rows.

```
jan = pd.DataFrame({"id": [1, 2],
                    "amt": [10, 20]})
feb = pd.DataFrame({"id": [3, 4],
                    "amt": [30, 40]})
```

Expected output:

[1]

	id	amt
0	1	10
1	2	20
0	3	30
1	4	40

[2]

	id	amt
0	1	10
1	2	20
2	3	30
3	4	40

Combining Data · Level 1

2 Glue columns side by side

- Put left and right side by side.

```
left  = pd.DataFrame({"a": [1, 2]},
                     index=[0, 1])
right = pd.DataFrame({"b": [9, 8]},
                     index=[5, 6])
```

Expected output:

[1]

	a	b
0	1.0	NaN
1	2.0	NaN
5	NaN	9.0
6	NaN	8.0

Combining Data · Level 2

3 inner then left

- Merge a and b on id (inner).
- Merge them with a left merge.

```
a = pd.DataFrame({"id": [1, 2, 3],
                  "n":  ["x", "y", "z"]})
b = pd.DataFrame({"id": [1, 3],
                  "tier": ["G", "S"]})
```

Expected output:

[1]

	id	n	tier
0	1	x	G
1	3	z	S

[2]

	id	n	tier
0	1	x	G
1	2	y	NaN
2	3	z	S

24 Keys with different names

- Merge the sales with the catalogue (left).
- Select the catalogue rows that were never sold.

```
s = pd.DataFrame({"sku": ["A", "B"],
                  "qty": [1, 2]})
c = pd.DataFrame({"code": ["A", "B", "Z"],
                  "name": ["pen", "bag",
                           "hat"]})
```

Expected output:

[1]

	sku	qty	code	name
0	A	1	A	pen
1	B	2	B	bag

[2]

	code	name
2	Z	hat

Combining Data · Level 3

5 The merge that invents rows

- Merge left and right on key.

```
left  = pd.DataFrame({"key": ["a", "b"],
                      "v": [1, 2]})
right = pd.DataFrame({"key": ["a", "a", "b"],
                      "w": [10, 20, 30]})
```

Expected output:

[1]

	key	v	w
0	a	1	10
1	a	1	20
2	b	2	30

Combining Data · Level 3

6 Four ways to join

- Join r onto l (the default left join).
- Join them with how='inner'.
- Join them with how='outer'.

```
l = pd.DataFrame({"a": [1, 2, 3]},
                 index=["x", "y", "z"])
r = pd.DataFrame({"b": [10, 20, 30]},
                 index=["y", "z", "w"])
```

Expected output:

[1]

	a	b
x	1	NaN
y	2	10.0
z	3	20.0

[2]

	a	b
y	2	10
z	3	20

[3]

	a	b
w	NaN	30.0
x	1.0	NaN
y	2.0	10.0
z	3.0	20.0